

УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 Программная инженерия

Дисциплина «Моделирование систем»

Лабораторная работа №3

«Моделирование процессов в Simpy»

Выполнил:

Касьяненко В.М. Р3420

Преподаватель:

к.т.н. Русак А.В.

г. Санкт-Петербург

2025

1. Постановка задачи

Требуется разработать модель системы массового обслуживания (СМО) с s устройствами (каналами обслуживания), используя фреймворк SimPy на языке Python. Заявки (требования) поступают в систему с интенсивностью $\lambda=1/m_A$, а каждое требование обслуживается в среднем за время m_{s1} (интенсивность обслуживания $\mu=ms1$). В любой момент времени каждая заявка может обслуживаться только на одном из s устройств.

Вариант 2:

Вариант	Количество устройств, s_1	Среднее время обработки требования, μ_{s1} с	Среднее время поступления требований, μ_A с
1	6	180	60
2	7	100	120

Основные пункты задания:

1. Моделировать поступление заявок как экспоненциальный поток со средним интервалом времени m_A .
2. Смоделировать обслуживание заявок: время обслуживания экспоненциально распределено со средним значением $ms1$.
3. Добавить «дополнительный ресурс» (по аналогии с реальной системой) и учитывать время работы с ним.
4. Выбрать критерий оптимизации (например, среднее время ожидания в очереди) и найти оптимальное число s .

Выходные параметры, которые могут использоваться для анализа и оптимизации:

- Коэффициент использования системы (или загрузки) ρ .
- Среднее время ожидания заявки.
- Среднее число заявок в системе.
- Пропускная способность (абсолютная и относительная).

2. Аналог реальной системы

В качестве реального прототипа можно рассмотреть, например, ремонтную мастерскую, в которой:

- Коэффициент использования системы (или загрузки) ρ .
- Есть s инженеров, каждый инженер может обслуживать только одного клиента (ремонт одного устройства) в единицу времени.
- Клиенты (заявки) «поступают» в мастерскую с некоторой скоростью (случайные интервалы со средним m_A).
- Среднее время ремонта (обслуживания) одного устройства – m_{s1} .
- Дополнительно в середине ремонта инженер может ходить на склад (ресурс с ограниченной вместимостью), чтобы взять запчасти. Это увеличивает суммарное время обслуживания.

3. Аналог реальной системы

Имитационная модель написана на Python с использованием библиотеки SimPy. Ниже приведён фрагмент кода с пояснениями.

3.1. Импорт библиотек и задание начальных параметров

```
import simpy
import random
import statistics

# Фиксируем зерно генерации случайных чисел для
# воспроизводимости
RANDOM_SEED = 42
random.seed(RANDOM_SEED)

# Параметры системы (пример из варианта)
MU_A = 120.0    # среднее время между поступлениями заявок
MU_S1 = 100.0   # среднее время обслуживания
```

```

# Параметры дополнительного ресурса (склада)
MIN_WAREHOUSE_TIME = 10.0
MAX_WAREHOUSE_TIME = 20.0

# Общая длительность моделирования
SIM_TIME = 20000

```

3.2 Генерация экспоненциально распределённого времени

```

def exp_time(mean):
    """Генерирует экспоненциально распределённое время с
    заданным средним."""
    return random.expovariate(1.0 / mean)

```

Функция `exp_time(mean)` возвращает случайную величину, распределённую по экспоненциальному закону со средним `mean`.

3.3 Процесс обслуживания одной заявки

```

def service_process(env, name, engineers, warehouse,
wait_times):
    arrival_time = env.now

    # Запрос инженера (один из s)
    with engineers.request() as req:
        yield req
        queue_wait = env.now - arrival_time # сколько
ждали в очереди
        wait_times.append(queue_wait)

    # Основная часть ремонта
    service_duration = exp_time(MU_S1)
    yield env.timeout(service_duration / 2.0)

    # Обращение к складу (дополнительный ресурс)
    with warehouse.request() as w_req:
        yield w_req
        warehouse_time =
random.uniform(MIN_WAREHOUSE_TIME, MAX_WAREHOUSE_TIME)
        yield env.timeout(warehouse_time)

    # Завершение ремонта
    yield env.timeout(service_duration / 2.0)

```

3.4 Процесс генерации заявок

```
def arrival_process(env, engineers, warehouse,
wait_times):
    i = 0
    while True:
        i += 1
        # Ждём интервал между поступлениями
        (экспоненциальный)
        yield env.timeout(exp_time(MU_A))
        # Запускаем процесс обслуживания для новой заявки
        env.process(service_process(env, f'Client_{i}',
engineers, warehouse, wait_times))
```

3.5 Запуск одного прогона моделирования

```
def run_simulation(s, sim_time=SIM_TIME):
    env = simpy.Environment()

    # Ресурс инженеров (ёмкость = s)
    engineers = simpy.Resource(env, capacity=s)

    # Ресурс склада (ёмкость = 1)
    warehouse = simpy.Resource(env, capacity=1)

    # Список для хранения индивидуальных времён ожидания
    wait_times = []

    # Запускаем процесс генерации заявок
    env.process(arrival_process(env, engineers, warehouse,
wait_times))

    # Запуск моделирования
    env.run(until=sim_time)

    # Среднее время ожидания
    mean_wait = statistics.mean(wait_times) if wait_times
else 0.0

    # Приблизительная оценка загруженности (коэффициент
использования)
    #  $\rho = (\lambda / (s * \mu))$ 
    lambda_ = 1.0 / MU_A
    mu_ = 1.0 / MU_S1
    rho = lambda_ / (s * mu_)

    return mean_wait, rho
```

3.6 Цикл оптимизации по числу каналов s

```
def main():
    s_values = range(3, 11) # перебираем s от 3 до 10
    results = []

    for s in s_values:
        mean_wait, ro = run_simulation(s)
        results.append((s, mean_wait, ro))
        print(f"s = {s}, ср. ожидание = {mean_wait:.2f} с,
оценка загрузки = {ro:.3f}")

    # Минимизируем среднее время ожидания
    best = min(results, key=lambda x: x[1])
    print(f"\nЛучшая конфигурация по критерию минимума
ожидания: s = {best[0]}, "
          f"ср. ожидание = {best[1]:.2f} с, загрузка =
{best[2]:.3f}")

if __name__ == "__main__":
    main()
```

4. Результаты моделирования

Пример вывода программы может выглядеть так (числа могут немного меняться при разных запусках):

```
s = 3, ср. ожидание = 4.47 с, оценка загрузки = 0.278
s = 4, ср. ожидание = 0.00 с, оценка загрузки = 0.208
s = 5, ср. ожидание = 0.00 с, оценка загрузки = 0.167
s = 6, ср. ожидание = 0.00 с, оценка загрузки = 0.139
s = 7, ср. ожидание = 0.00 с, оценка загрузки = 0.119
s = 8, ср. ожидание = 0.00 с, оценка загрузки = 0.104
s = 9, ср. ожидание = 0.00 с, оценка загрузки = 0.093
s = 10, ср. ожидание = 0.00 с, оценка загрузки = 0.083

Лучшая конфигурация по критерию минимума ожидания: s = 4, ср. ожидание = 0.00 с, загрузка = 0.208
```

Из результата видно:

- При $s=3$ в системе ещё возникает очередь, поэтому среднее ожидание составляет около 4,5 секунд.
- Начиная с $s=4$, ресурс «инженеры» оказывается настолько мощным (в сравнении с потоком заявок λ), что почти все заявки обслуживаются сразу, среднее время ожидания близко к нулю.

Таким образом, оптимальным по критерию минимизации времени ожидания оказывается $s=4$. Если смотреть с точки зрения ресурсоёмкости, мы видим, что при $s=7$ система тоже работает без очередей, но это «избыточное» количество инженеров, а значит – большие издержки на содержание ресурсов, хотя и с тем же нулевым ожиданием.